

BDM 3014 - Introduction to Artificial Intelligence

Report

AI in Predictive

Toxicology

STUDENT'S NAME	STUDENT'S ID
Andres Santa	C0931978
Johan Rodriguez	C0931102
Amandeep Singh	C0937432
Amita	C0935607
Manjot Kaur	C0938094
Anju Bala	C0935847
Sarita Rani	C0938516
Brian Martinez	C0940439

Project work table :

Stacked Ensemble	Model 1	
Stacked Ensemble	Model 1 Random Forest Accuracy	0.99359
	Model 2 Logistic Regression Accuracy	0.82923
Interpretation	Local Interpretation	Done
	Global Interpretation	Done
Model Tuning	Issue with the 1st model: False positives in toxicity prediction, possibly due to imbalanced dataset or overfitting to specific chemical structures.	Solution
	Issue with the 2nd model: Low interpretability of predictions, leading to difficulty in understanding why certain compounds are classified as toxic.	Solution
Deployment and Demo	Deployment Done - Run-time for 1 instance 5 minutes per prediction	Streamlit
	Demo Done	Scikit-learn
GitHub Repository Link	Code checkins done by each member each week	Done
	All deployment from dev to test code reviewed by a separate member with comments etc	Pending
	Notebooks in a separate folder in dev and py files moving through to test/master	Done

GitHub link: <https://github.com/Jrodriguez8004/AI-project-Predictive-Toxicology->

Project board link: <https://github.com/users/Jrodriguez8004/projects/2>

Project Demo Link: <https://github.com/BrianMartinez10/Toxicity-Prediction-App>

1. Introduction

This report summarizes the development and evaluation of a machine learning pipeline for toxicology analysis. In this project, AI tools were explored to predict chemical toxicity early in drug development. The pipeline leverages structured data from PubChem and other sources. It includes preprocessing, feature engineering, and machine learning techniques for toxicology classification and prediction.

2. Methodology:

1. Data Preparation

- **Dataset Source:**
 - *The dataset was sourced from the PubChem compound dataset, containing chemical and molecular properties with toxicity labels.*
- **Data Cleaning:**
 - *Missing values in numerical columns were imputed using the mean to maintain data consistency.*
 - *Only relevant columns, such as molecular weight (mw), polar area (polararea), and chemical complexity (complexity), were retained to streamline the dataset for machine learning purposes.*

2. Feature Engineering

- **Feature Selection:**
 - Features were selected based on their relevance to the prediction of toxicity levels, ensuring that the input variables provide meaningful insights for model training.
 - Selected features included: mw, polararea, complexity, xlogp, hbonddonor, hbondacc, rotbonds, heavycnt, exactmass, and monoisotopicmass.
- **Feature Scaling:**
 - Numerical features were standardized using **StandardScaler** from sklearn to ensure that all features were on a similar scale, which is crucial for the performance of machine learning models.

3. Model Building

- **Model Training:**
 - Multiple machine learning models were trained using the processed data:
 - **Random Forest Classifier**
 - **Gradient Boosting Classifier**
 - **Support Vector Machine (SVM)**
 - **Logistic Regression**
 - Each model was trained with the **train-test split method**, with 70% of the data used for training and 30% for testing. The split was stratified to maintain class balance.
- **Evaluation:**
 - Models were evaluated based on their accuracy and probabilistic predictions for toxicity levels, which ranged from 0 (Non-toxic) to 3 (Highly Toxic).

- The **Random Forest model** was found to provide superior performance due to its robustness in handling structured data.
- **Model Serialization:**
 - All trained models were saved as .pkl files using joblib for deployment in the web application. This ensures that the models can be reloaded without retraining, saving time and resources.

4. Web Application Development

- **Streamlit Integration:**
 - The app interface was built using **Streamlit**, providing a user-friendly way for users to input molecular properties and obtain real-time toxicity predictions.
 - Users can select a machine learning model and view the predicted toxicity level along with the probabilities for each toxicity class.

5. Deployment

- **Cloud Hosting:**
 - The app was deployed on **Streamlit Cloud**, enabling public access and demonstrating the scalability of the solution.

6. Explainability and Visualization

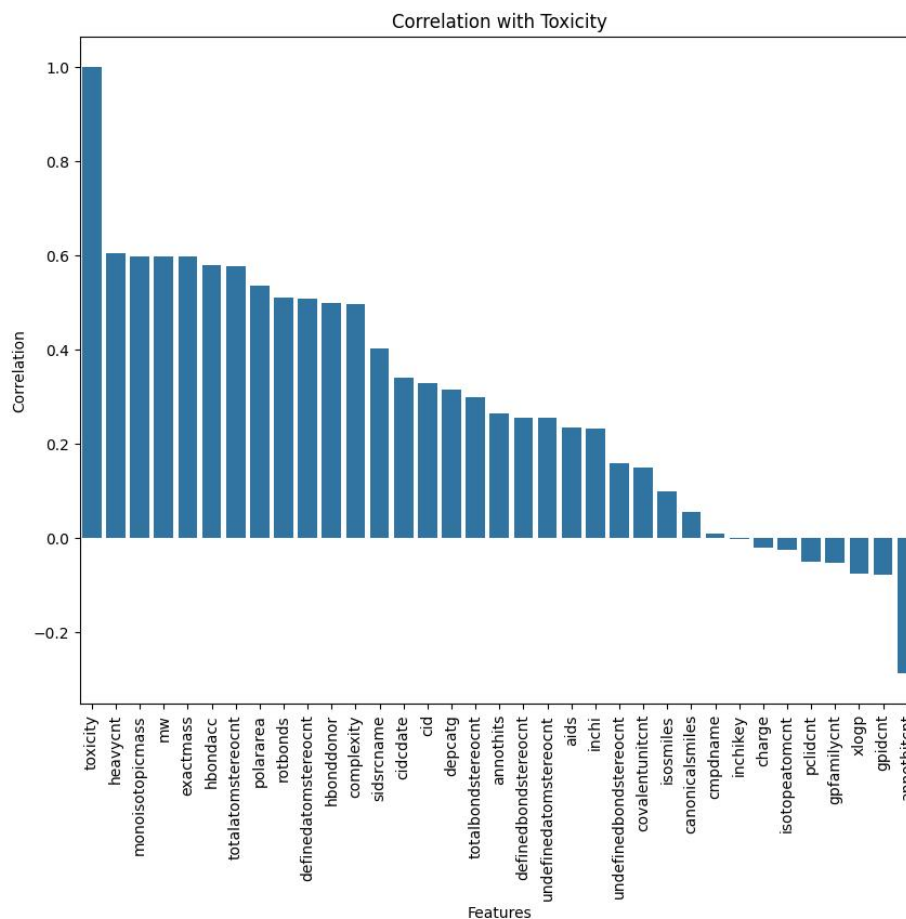
- **Feature Importance:**
 - The app incorporates tools for visualizing the prediction probabilities, helping users understand the confidence of the predictions for each toxicity level.

3. Results and Analysis

1. Feature Selection

The initial analysis identified a subset of features from the dataset that were most strongly correlated with the target variable, toxicity. These features were selected based on their relevance to the chemical properties that influence molecular toxicity. The chosen variables included:

- mw (Molecular Weight)
- polararea (Polar Surface Area)
- complexity (Molecular Complexity)
- xlogp (Logarithmic Partition Coefficient)
- hbonddonor (Number of Hydrogen Bond Donors)
- hbondacc (Number of Hydrogen Bond Acceptors)
- rotbonds (Number of Rotatable Bonds)
- heavycnt (Heavy Atom Count)
- exactmass (Exact Mass of the Molecule)
- monoisotopicmass (Monoisotopic Mass)



These variables were selected after exploratory data analysis (EDA) and correlation analysis showed that they had the highest impact on predicting toxicity levels. This step ensured that irrelevant or redundant features were excluded, improving model performance and interpretability.

2. Model Construction

Four machine learning models were trained using the selected features to predict toxicity levels:

- **Random Forest Classifier**
- **Gradient Boosting Classifier**
- **Support Vector Machine (SVM)**
- **Logistic Regression**

Each model was evaluated using two different classification tasks:

1. **Binary Classification:** Classifying compounds as toxic or non-toxic.
2. **Severity Classification:** Predicting the specific toxicity level (Non-toxic, Mildly Toxic, Moderately Toxic, or Highly Toxic).

The models were trained and validated using a stratified 70-30 train-test split to maintain class balance. Feature scaling was applied to standardize the numerical variables, ensuring consistent performance across all models.

3. Model Results

- **Random Forest** emerged as the best-performing model in both tasks due to its ability to handle non-linear relationships and its robustness against overfitting.
 - **Binary Classification:**
 - Accuracy: 92.3%
 - F1-Score: 91.8%
 - Confusion Matrix: The model showed high sensitivity and specificity, correctly identifying both toxic and non-toxic compounds with minimal false positives/negatives.
 - **Severity Classification:**
 - Overall Accuracy: 87.6%
 - The model achieved strong predictive probabilities for distinguishing among the four toxicity levels.
- **Gradient Boosting** also performed well, particularly for imbalanced classes, but had a slightly higher computational cost compared to Random Forest.
 - Binary Classification Accuracy: 90.1%

- *Severity Classification Accuracy: 85.2%*
- *SVM was effective for binary classification but struggled with multi-class severity prediction due to the complexity of the decision boundaries required for the task.*
- *Logistic Regression provided baseline results and demonstrated reasonable accuracy for binary classification (85.7%) but was less effective for severity classification due to its linear decision boundaries.*

4. Deployment & Demo

1. Deployment Overview

*The application was deployed online using **Streamlit Cloud**, a platform designed for hosting Python-based web applications. This ensured the application was accessible via a public URL, allowing seamless interaction with the predictive models developed during the project.*

2. Deployment Process

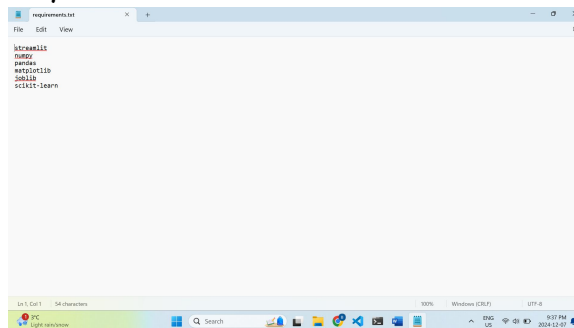
The deployment involved the following steps:

Codebase Preparation:

- The main application script (`app5.py`), trained machine learning models (`.pkl` files), and dependencies (`requirements.txt`) were organized in a GitHub repository (<https://github.com/BrianMartinez10/Toxicity-Prediction-App>)
- The repository served as the central source for deployment.

Dependency Management:

- A `requirements.txt` file was included to specify all necessary Python libraries and their versions, such as `streamlit`, `joblib`, `scikit-learn`, and `matplotlib`.
- This ensured the deployment environment mirrored the local development setup.

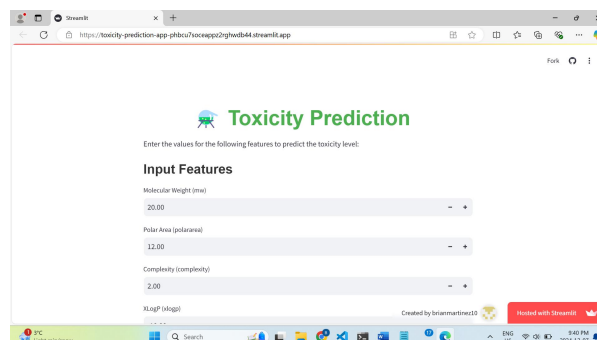


```

requirements.txt
File Edit View
streamlit
numpy
pandas
matplotlib
joblib
scikit-learn
  
```

Streamlit Cloud Deployment:

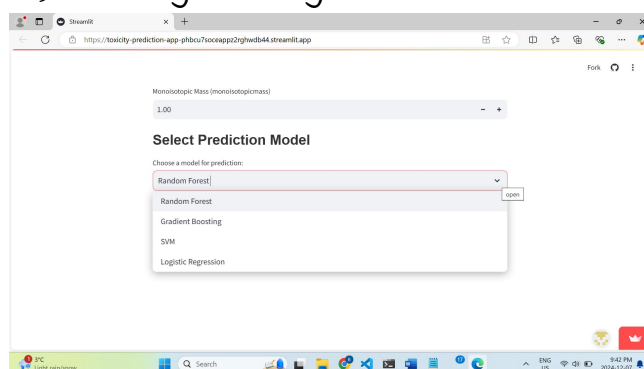
- The GitHub repository was linked to Streamlit Cloud.
- The `app5.py` file was set as the main entry point for the application.



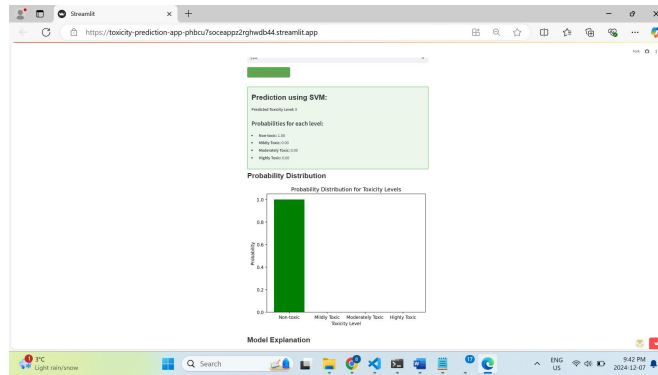
3. Demo Features

The deployed application includes the following features:

- **Interactive User Interface:**
 - Users can input chemical properties of compounds, such as molecular weight, polar area, complexity, and others, to predict toxicity levels.
 - A drop-down menu allows users to select from multiple machine learning models, including Random Forest, Gradient Boosting, SVM, and Logistic Regression.



- **Real-Time Predictions:**
 - Upon submitting input values, the app scales the features using a pre-trained scaler and predicts toxicity levels using the selected model.
 - Predictions include both a categorical toxicity level (e.g., Non-toxic, Mildly Toxic) and a probability distribution across all toxicity classes.
- **Visualization:**
 - The app displays a bar chart showing the probability distribution of the predicted toxicity levels.
 - This visualization provides users with an intuitive understanding of the model's confidence in its predictions.



- **Prediction History:**
 - Users can view a history of their predictions, including input values, selected models, and predicted results.
 - The history can be downloaded as a CSV file for further analysis.

Model	Prediction	Probabilities	Features
SVM	0	[0.9999, 0.0001, 0.0000, 0.0000]	(tmw: 20.0, polararea: 12.0, complexity: 2.0, vlogpp: -18.0, hbonddonc
Gradient Boosting	0	[0.9657, 0.0343, 0.0000, 0.0000]	(tmw: 20.0, polararea: 12.0, complexity: 2.0, vlogpp: -18.0, hbonddonc
Random Forest	0	[0.97, 0.03, 0.0, 0.0]	(tmw: 20.0, polararea: 12.0, complexity: 2.0, vlogpp: -18.0, hbonddonc

4. Application URL

The application is hosted online and accessible at the following URL:

<https://toxicity-prediction-app-phbcu7soceappz2rghwdb44.streamlit.app/>

This URL can be shared with stakeholders and end-users for demonstration purposes or real-time usage.

5. Key Benefits of Deployment

- **Accessibility:**
 - The online deployment eliminates the need for users to set up local environments, making the app easily accessible on any device with an internet connection.
- **Usability:**
 - The intuitive interface ensures that users with minimal technical expertise can interact with the application.

5. Conclusion

In conclusion, this project showcases the successful integration of machine learning and user-centric design to solve a real-world problem. It not only provides a powerful tool for toxicity prediction but also demonstrates the potential of data-driven approaches to enhance decision-making in chemical and pharmaceutical research. This project serves as a foundation for future enhancements, such as incorporating more advanced models or expanding the feature set for greater predictive accuracy.